Climbing Grade Converter — Part 1: Core Logic

Overview

Climbers across different countries and disciplines use different scales to rate route difficulty. Build a service that can translate between them.

Your job in this first part is to implement the **core conversion logic** — no web layer, no database, no API. Just clean, well-structured code that converts grades between systems.

Supported Grading Systems

Your converter must support the following five systems:

System	Used For	Region	Example Grades
French	Sport / Lead climbing	Europe	4a, 5c, 6a, 6a+, 7b, 8a, 9a
YDS (Yosemite Decimal System)	Sport / Lead / Trad climbing	North America	5.6, 5.9, 5.10a, 5.10d, 5.12a, 5.14a
UIAA	Sport / Trad climbing	Central & Eastern Europe	IV, V+, VI, VII-, VIII, X, XI
V-Scale (Hueco)	Bouldering	North America	VB, V0, V1, V4, V8, V12, V17
Fontainebleau (Font)	Bouldering	Europe	3, 4, 5+, 6A, 6C+, 7A, 8A, 8C+

What You Need to Build

1. Convert a Grade

Given a grade value, a source system, and a target system, return the equivalent grade.

Example:

- Input: value = "6a", from = FRENCH, to = YDS → Output: "5.10b"
- Input: value = "V5", from = V_SCALE, to = FONT → Output: "6C"
- Input: value = "5.12a", from = YDS, to = UIAA → Output: "VIII+"

2. List Available Grading Systems

Return all supported grading systems with their name, code, category (route or boulder), and a short description.

3. Get the Full Grade Table for a System

Given a system, return all its grades along with their equivalents in every compatible system.

Example: Asking for the French table should return every French grade with its YDS and UIAA equivalents.

Rules

- **French, YDS, and UIAA** are all **route/sport climbing** grades. They can be converted between each other.
- **V-Scale and Fontainebleau** are both **bouldering** grades. They can be converted between each other.
- **You cannot convert between route grades and boulder grades.** Converting V5 to French or 6a to V-Scale makes no sense — this should be rejected with a clear error.
- Grade matching should be **case-insensitive** (e.g., "6a", "6A", and "6a+" should all work).
- Invalid or unrecognized grades should produce a clear error.
- Unknown or unsupported system names should produce a clear error.

Grade Conversion Reference

Use the following approximate mappings. Conversions between grading systems are never exact — these are the commonly accepted equivalents.

Route / Sport Climbing Grades

French	YDS	UIAA
3	5.4	III
4a	5.5	IV
4b	5.6	IV+
4c	5.7	V
5a	5.8	V+
5b	5.9	VI-
5c	5.10a	VI
6a	5.10b	VI+
6a+	5.10c	VII-
6b	5.10d	VII
6b+	5.11a	VII+
6c	5.11b	VII+/VIII-
6c+	5.11c	VIII-
7a	5.11d	VIII
7a+	5.12a	VIII+
7b	5.12b	VIII+/IX-
7b+	5.12c	IX-
7c	5.12d	IX
7c+	5.13a	IX+
8a	5.13b	IX+/X-
8a+	5.13c	X-
8b	5.13d	X

French	YDS	UIAA
8b+	5.14a	X+
8c	5.14b	X+/XI-
8c+	5.14c	XI-
9a	5.14d	XI
9a+	5.15a	XI+
9b	5.15b	XI+/XII-
9b+	5.15c	XII-
9c	5.15d	XII

Bouldering Grades

V-Scale	Fontainebleau
VB	3
V0	4
V1	5
V2	5+
V3	6A
V4	6B
V5	6C
V6	7A
V7	7A+
V8	7B+
V9	7C
V10	7C+
V11	8A

V-Scale	Fontainebleau
V12	8A+
V13	8B
V14	8B+
V15	8C
V16	8C+
V17	9A

What You'll Practice

- Structuring code into clean layers (services, models, enums)
- Working with enums and static data mappings
- Input validation and meaningful error handling
- Thinking about edge cases
- Writing code that's easy to extend later (new systems, new endpoints on top)

Bonus (Optional)

- Support a "convert to all" mode — given a grade and source system, return equivalents in **all** compatible systems at once.
- Support partial matching — if someone passes "5.10" without a letter suffix, return all sub-grades (5.10a through 5.10d).

Coming Up Next

This is just the beginning. Once you nail the core logic, we'll keep building on top of it:

- **Part 2 — Unit Tests:** Write tests for your conversion logic, edge cases, and error handling.
- **Part 3 — REST API:** Expose your logic through HTTP endpoints with proper status codes, request validation, and error responses.

- **Part 4 — Database:** Move the grade data from static code into a real database (PostgreSQL) using an ORM.
- **Part 5 — UI:** Build a simple frontend so users can actually use the converter in a browser.
- **Part 6 — Docker:** Containerize the whole thing (app + database) with Docker Compose so it runs anywhere.
- **Part 7 — CI/CD:** Set up a pipeline that runs your tests and builds your app automatically on every push.

Don't worry about any of this yet — just focus on Part 1. But write your code knowing that all of this is coming. Clean structure now saves you pain later.

Good luck, and send it! 🧗